

CSCE 627 THEORY OF COMPUTABILITY · MODULE 09 · LECTURE NOTES

Computable Functions

The positive theory, and a theorem about self-reference.

MODULE OUTCOMES

Define primitive recursion and explain what it cannot express.

Explain μ -recursion and why it recovers full power.

State the recursion theorem and use it.

Explain Kolmogorov complexity and its uncomputability.

Explain the busy beaver function and what it demonstrates.

1 Primitive recursion

PRIMITIVE RECURSIVE FUNCTIONS ALWAYS TERMINATE, AND MISS SOME THAT DO

Build functions from the constant zero, the successor function, and the projections, using composition and recursion on a single argument that decreases. That is the primitive recursive class.

Every primitive recursive function is total — it terminates on every input, by construction, since the recursion always decreases a well-founded measure — **so termination is decidable for this class**, trivially, which is Module 05 §4's restriction response in its purest form.

And it is exactly the class of bounded loops: every primitive recursive function corresponds to a program whose loop bounds are computed in advance of entering the loop. **No while, only for with a precomputed count.**

But it misses total computable functions. Ackermann's function is total, obviously computable, and grows faster than every primitive recursive function — established by a diagonal argument over the class (§1's code), which is Module 05's method applied to a restricted class rather than to all machines.

```

THE PRIMITIVE RECURSIVE functions can be enumerated:
f_0, f_1, f_2, ...

Define g(n) = f_n(n) + 1.

g is total and computable -- enumerate until you reach the
nth, run it on n, add one.
And g differs from every f_n at the input n.
So g is NOT primitive recursive.
    -- Module 05's diagonalisation, applied to a CLASS
       rather than to all machines.

ACKERMANN'S FUNCTION is the standard concrete witness, and
it grows unmanageably fast:
    A(1,n) is about 2n
    A(2,n) is about 2^n
    A(3,n) is a tower of 2s of height n
    A(4,2) already exceeds the number of particles in the
        observable universe

THE LESSON: TOTALITY CANNOT BE GUARANTEED BY SYNTAX WITHOUT
LOSING EXPRESSIVENESS.
    -- which is exactly the trade a total functional
       language makes (Agda, Idris, Coq's terminating
       fragment): every program provably terminates, and
       some computable total functions cannot be
       written.

```

Total languages are strictly less expressive, provably — which is a precise statement of what a **termination guarantee costs**, and a more useful framing than the usual informal complaint that total languages are awkward. **The awkwardness is a theorem**, and the question for a language designer is whether the functions lost are ones anyone wanted — which, in practice, they mostly are not.

2 Recovering full power

M-RECURSION ADDS UNBOUNDED SEARCH, AND THAT IS EXACTLY THE MISSING PIECE

Add the μ operator: $\mu y.R(x,y)$ is the least y such that $R(x,y)$ holds. Search upward from zero, testing each candidate, forever if necessary.

The resulting class — the μ -recursive functions — is exactly the Turing-computable functions, which is Module 01 §2's coincidence of independent formalisations in this particular form.

And the μ operator is precisely where non-termination enters — if no y satisfies R , the search runs forever, and nothing else in the definition can fail to terminate.

So full computational power costs exactly one unbounded while loop. The entire undecidability of this subject is traceable to that single construct — which is a remarkably clean localisation, and it says something about language design: **the construct that makes a language Turing-complete is the same construct that makes its programs undecidable**. You cannot have one without the other.

3 The recursion theorem

THE RECURSION THEOREM: for any computable function t , there is a program p with

the behaviour of $p = t(\text{description of } p)$

In other words: you may write a program that uses its own source code, and such a program is guaranteed to exist.

WHY THIS IS NOT OBVIOUS: you cannot simply embed your own source text inside yourself -- that is circular, because embedding the text makes the text longer, which changes what must be embedded. The theorem says a fixed point exists regardless.

CONSEQUENCES

QUINES exist -- programs that print their own source.

Not a party trick; a corollary of a theorem.

VIRUSES can copy themselves, for the same reason.

MODULE 05's PROOF becomes cleaner: build a machine that obtains its own description and then contradicts the decider, with no diagonal construction needed at all.

RICE'S THEOREM has a short proof from it.

AND THE Y COMBINATOR is the lambda-calculus version -- recursion obtained in a language that has no recursion construct (Module 11 section 1).

SELF-REFERENCE IS A THEOREM, NOT A LOOPHOLE.

Self-reference is guaranteed rather than merely permitted — which is why Module 05's proof is not a trick (the objection answered there gets its formal answer here) **and why quines are inevitable rather than ingenious. Students generally find the recursion theorem reassuring after Module 05**, because it converts an uncomfortable construction into a routine application of a general result.

4 Describing things, and growing fast

KOLMOGOROV COMPLEXITY: THE LENGTH OF THE SHORTEST PROGRAM THAT PRINTS IT

$K(x)$ is the length of the shortest program that outputs x and halts. A string is *random* if $K(x)$ is approximately $|x|$ — that is, if it has no description shorter than itself. Which is a definition of randomness for an individual object, something probability theory cannot provide.

Most strings are random, by counting: there are fewer programs shorter than n bits than there are strings of length n , so most strings cannot have short descriptions. **So incompressibility is the norm and compressibility the exception** — which is the information-theoretic reason general-purpose compression works on real files and not on arbitrary ones.

And K is uncomputable — by the Berry paradox made precise. A program that computed K could find 'the smallest number not describable in fewer than a thousand bits', which it has just described in far fewer. **Formally: a K -computing program would itself be a short description of outputs it certifies as having no short description.**

Which gives a one-line proof that no general lossless compressor can shrink every input (most inputs are incompressible) **and a formal definition of randomness that does not mention probability at all** — both of which are worth having, and the second connects to CSCE 658's derandomisation questions.

THE BUSY BEAVER FUNCTION GROWS FASTER THAN ANYTHING COMPUTABLE

$BB(n)$ is the largest number of steps that any halting n -state Turing machine takes before halting. A perfectly well-defined function of n — there are finitely many n -state machines, so the maximum over the halting ones exists.

And it is uncomputable — if you could compute $BB(n)$, you could decide halting for n -state machines by running a machine for $BB(n)$ steps and concluding it never halts if it has not. So BB 's uncomputability is immediate from Module 05.

It grows faster than every computable function, which is strictly stronger than merely being uncomputable — it eventually exceeds any computable function you care to name.

And the known values stop almost immediately: $BB(1)$ through $BB(4)$ are small and known, $BB(5) = 47,176,870$ was established in 2024 after decades of effort by a distributed collaboration, and $BB(6)$ is known to exceed 10 to a tower of 15 — while $BB(748)$ is independent of standard set theory, because a 748-state machine can search for a contradiction in ZFC. **A question about one specific finite number that no amount of mathematics can settle**, which is the most striking fact in this module and a good bridge to Module 10.

- **Totality guaranteed by syntax costs expressiveness, provably** — which is what a total language trades (§1).
- **Full power costs exactly one unbounded search**, and all of the subject's undecidability traces to it (§2).
- **Self-reference is a theorem**, so Module 05's construction was inevitable rather than clever (§3).
- **Randomness has a definition that does not mention probability**, and that definition is uncomputable (§4).

- **And uncomputable functions can be perfectly concrete — $BB(5)$ is a specific integer that took sixty years to determine**, which is a useful corrective to the impression that uncomputability is about exotic objects.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — sections 6.1–6.2 and 6.4	The recursion theorem of §3 and an introduction to §4's Kolmogorov complexity.
Li & Vitányi — An Introduction to Kolmogorov Complexity and Its Applications	The reference for §4 , including the randomness definition and its applications. Library copy.
Aaronson — The Busy Beaver Frontier (free)	§4's busy beaver material , including the independence results and the state of knowledge. Unusually readable.
The bbchallenge collaboration (free)	The $BB(5)$ proof, completed in 2024 , with the whole effort documented — a good example of what settling one value of an uncomputable function actually requires.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write addition, multiplication, and exponentiation** as primitive recursive definitions.
- 2 **Implement Ackermann's function** and compute $A(3,n)$ for n up to a limit your machine tolerates.
- 3 **Write out the diagonal argument** showing a total computable function that is not primitive recursive.
- 4 **Express a μ -recursive function** that is not primitive recursive, and identify where the unbounded search is.
- 5 **Write a quine** in a language of your choice.
- 6 **Explain, using the recursion theorem, why that was possible.**
- 7 **Re-prove the halting problem using the recursion theorem** and compare against Module 05's proof.
- 8 **Estimate $K(x)$ for three strings** by compressing them, and explain why compression is only an upper bound.
- 9 **Prove no compressor shrinks every input**, by counting.
- 10 **Look up the $BB(5)$ result** and summarise what the proof had to establish.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define primitive recursion and say what programs it corresponds to.
- 2 Why is every primitive recursive function total?
- 3 Show that it misses a total computable function.
- 4 What does a total functional language trade, and is that a theorem?
- 5 What does μ add, and what class results?
- 6 Where does non-termination enter, and what follows?
- 7 State the recursion theorem and say why it is not obvious.
- 8 Name five of its consequences.
- 9 Define Kolmogorov complexity and explain why it is uncomputable.
- 10 Define $BB(n)$, prove it uncomputable, and say what is known.